

ポートフォリオ

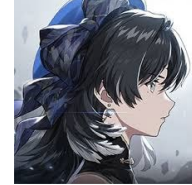
情報文化学園アーツカレッジヨコハマ
ゲームクリエイター学科プログラマーコース
西結斗

自己紹介

使えるスキル・ツール

- ・ プログラム言語
C・**C++**・**C#**・**VBA**
- ・ ゲームエンジン
Unity
- ・ バージョン管理ツール
SVN・Gitを使用した開発
(TortoiseSVN・TortoiseGitを使用)
- ・ その他
Word・PowerPoint・Excel
PhotoShop・Illustrator
Effekseer・VisualStudio2022

好きなゲーム



最近遊んだゲーム



ゲーム制作における姿勢

自分が納得するまでゲーム制作を進め、ユーザーの操作感にこだわって制作をしていきました。
また、演出面にもこだわりを入れてきて、ユーザーの気持ちを上げ下げさせるように作成してきました。

作品概要

SKY WARS



・ジャンル

「戦闘3Dアクションゲーム」
ジャスト回避を主体としたアクションゲームになっています。

・制作環境

VisualStudio2022

C++

DxLib

・制作期間

ジャスト回避の演出など基本アクションを6月から個人で作り始め
10月から本格的にチームでバトルアクションゲームを制作しました。

・制作体制

チーム制作で作っており、プログラマー2人、デバッカー1人で制作
しています。

ゲーム制作にあたり重視した点

目標

戦闘をしていて楽しく見栄えが良いものを制作する。



自分が注視してきた点

データの外部化
気持ちよさの演出
ボスの攻撃時における工夫
ロックオンカメラ

私は、**ジャスト回避**や、そこからの攻撃を主に注力をしていき、ジャスト回避の演出やジャスト回避をしやすくさせることに注力をしていきました。

また**カットシーン**などといったゲームフローの中にも演出を組み込むことによりゲーム体験の没入感を上げていくことにも力を入れて制作をしていきました。

またそれらをプログラマー以外でも制作しやすくするために**データの外部化**や**ゲーム内で調整が出来るようにするツール**に着目をして積極的に外部化をさせていきました。

データの外部化

データの外部化

今まではデータをプログラム内に収めていましたが、それではパラメーターの調整がしにくいと感じましたので、データの外部化に力を入れていきました。以下は外部化したデータの項目です。

プレイヤーのパラメーター

ステージのパラメーター

ボスのパラメーター

カットシーンのパラメーター

プレイヤーのパラメーター

攻撃時における移動量、
ダメージ量
当たり判定の生成時間、
多段ヒットの設定、
モーションキャンセルが可能になる時間、
次の派生攻撃のID、
などをjsonデータをいじることによって、
簡単にプレイヤーの攻撃の調整が行える
ようになりました。

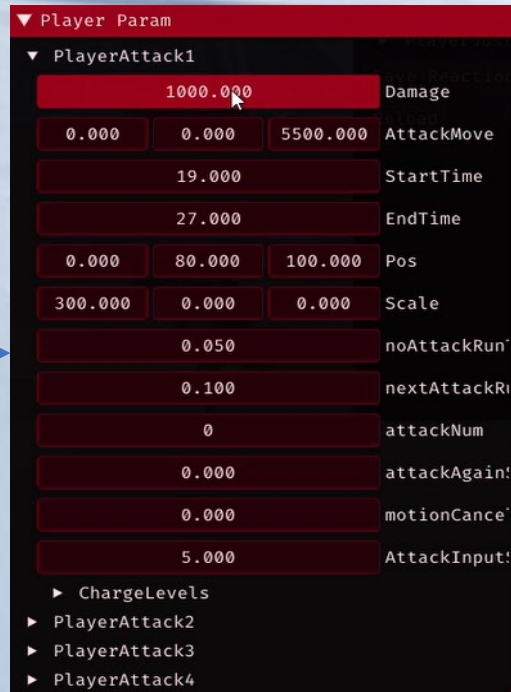
```
"AttackMove": { //攻撃時にどれくらい移動するか
  "x": 0.0,
  "y": 0.0,
  "z": 5500.0
},
"PlayerState": "PlayerAttack1", //ステートのID
"attackAgainStartCounterMax": 0.0, //多段ヒットするときの攻撃間隔
"attackInputStartTime": 5.0, //先行入力の開始時間
"attackNum": 0, //何回多段ヒットするか
"chargeLevels": [], //チャージするときにおけるチャージによってのパラメータ変更
"collTrans": { //当たり判定のトランスフォーム
  "position": {
    "x": 0.0,
    "y": 80.0,
    "z": 100.0
  },
  "rotation": {
    "x": 0.0,
    "y": 0.0,
    "z": 0.0
  },
  "scale": {
    "x": 300.0,
    "y": 0.0,
    "z": 0.0
  }
},
"collisionFinishTime": 27.0, //攻撃当たり判定終了フレーム
"collisionStartTime": 19.0, //攻撃当たり判定開始フレーム
"hitDamage": 500.0, //ダメージ量
"motionCancelStartTime": 0.0, //モーションキャンセルを何フレーム目から出来るか
"nextAttackRunTimer": 0.10000000149011612, //攻撃の後隙(次に攻撃するときの)
"noAttackRunTimer": 0.05000000074505806, //攻撃の後隙(次に攻撃しないときの)
"normalAttackNextID": "PlayerAttack2", //次の攻撃用ID(通常攻撃ボタンを押したときの)
"specialAttackNextID": "PlayerAttack4" //次の攻撃用ID(強攻撃ボタンを押したときの)
```


プレイヤーのパラメーター

ツールで変更前



ツールで調整をし、リロードボタンを押す

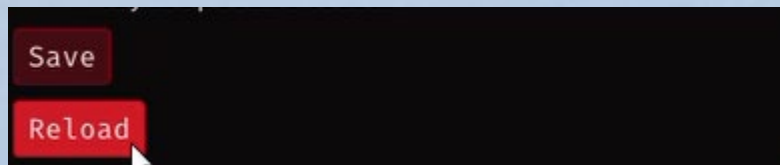


即時反映



Saveを押すとJsonデータに保存

Reloadを押すとゲーム内反映



またゲーム内で調整が可能となっており、jsonファイルを開かずともパラメータの調整が出来るようになりました。

ステージのパラメーター

ステージのIDの設定、
ステージの名前の設定、
マップのモデルの設定、
BGMの設定
登場するボスの種類
を設定しステージを作れます。

またjsonにデータを追加するだけでプログラムをいじることなくステージを追加出来るようにしました。

```
"id": 0,           //ステージID  
"name": "stage0", //ステージの名前  
"map": 0,         //マップのモデルID  
"bgm": "PlaySceneBGM", //BGMの名前  
"bossID": 0       //登場するボスのID
```

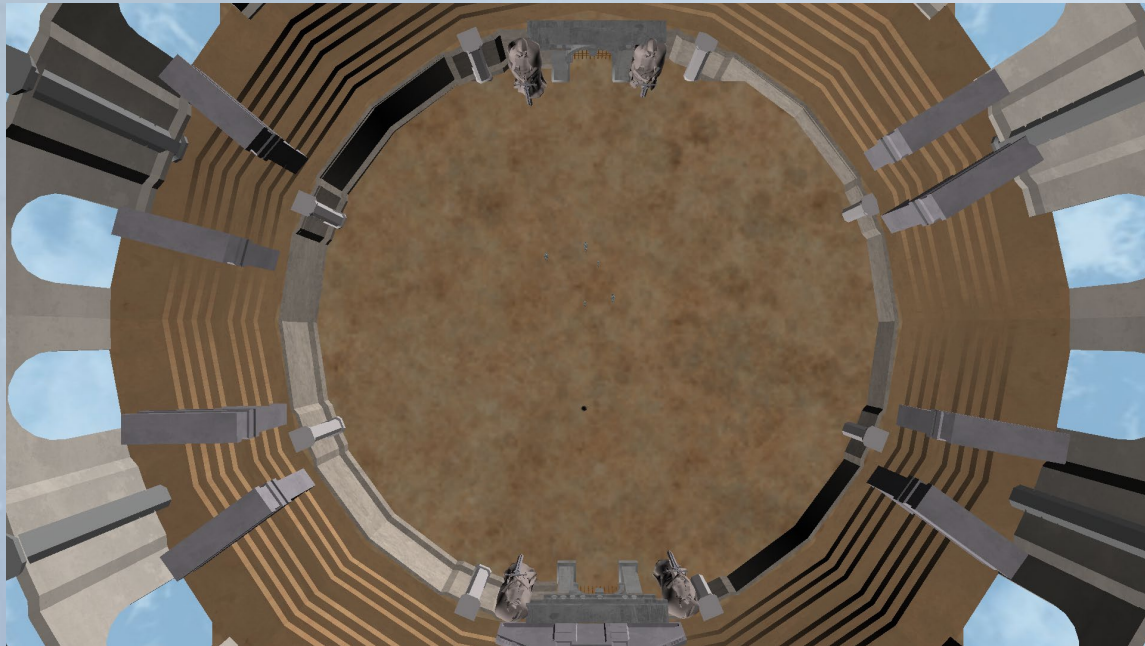


マップのモデルデータは別jsonファイルで管理

```
"id": 0,           //マップID  
"map": "stage_s", //マップのファイル名  
"Transform": {     //マップのトランスフォーム  
  "position": {  
    "x": 25000.0,  
    "y": -4000,  
    "z": 20000.0  
  },  
  "rotation": {  
    "x": 0.0,  
    "y": 0.0,  
    "z": 0.0  
  },  
  "scale": {  
    "x": 10.0,  
    "y": 10.0,  
    "z": 10.0  
  }  
}
```

ステージのパラメーター

ステージ0 & ステージ1で使用。



テストステージ1で使用。



このようにマップのデータをいじるだけでマップのモデルが簡単に変更が出来るようになっています。

ボスのパラメーター

基本パラメーター

ボスのID、防御力、HP、モデルの名前

これらは先ほどのStageのBossIDと結びついていて、StageのBossIDからこちらに参照する流れになっています。

```
"Boss": [  
  {  
    "bossID": 0,  
    "bossName": "",  
    "defense": 500.0,  
    "hp": 30000.0,  
    "modelName": "BossModel"  
  },  
  {  
    "bossID": 1,  
    "bossName": "",  
    "defense": 1000.0,  
    "hp": 30000.0,  
    "modelName": "BossModel2"  
  }  
]
```

攻撃振り分けパラメーター

攻撃のStateか、距離で確率UP、どれくらい確立をUPさせるか、攻撃のID、攻撃の重み

これらはボスがどの攻撃を出していくかを抽選するために必要なパラメーターとなっています。

```
"AttackSort": {  
  "BossBackStep": {  
    "addWeight": 0,  
    "attackState": false,  
    "distance": 0.0,  
    "id": "BossBackStep",  
    "maxAction": 0,  
    "priority": 50,  
    "weight": 30  
  },  
}
```


ボスのパラメーター

ボスが複数体いても耐えられるようなボスの攻撃データを作成しました。

ボスの種類ごとにフォルダを作る。

Boss1	2026/04/15 22:23	ファイル フォルダ	
BossParam.json	2026/04/16 16:32	JSON ソース ファイル	1 KB

Boss1のフォルダの中

フォルダの中に入っているすべてのjsonを読み込む。(それが攻撃の数になる)

Sorting	2026/04/16 14:05	ファイル フォルダ	
BossHalfSpecialAttack.json	2026/04/17 16:24	JSON ソース ファイル	4 KB
BossNormalAttack1.json	2026/04/17 16:24	JSON ソース ファイル	2 KB
BossNormalAttack2.json	2026/04/17 16:24	JSON ソース ファイル	2 KB
BossNormalAttack3.json	2026/04/17 16:24	JSON ソース ファイル	2 KB
BossNormalAttack4.json	2026/04/17 16:25	JSON ソース ファイル	2 KB
BossNormalAttack5.json	2026/04/17 16:25	JSON ソース ファイル	2 KB
BossNormalAttack6.json	2026/04/17 16:25	JSON ソース ファイル	2 KB
BossNormalAttack7.json	2026/04/17 16:25	JSON ソース ファイル	5 KB
BossNormalAttack8.json	2026/04/17 16:25	JSON ソース ファイル	5 KB
BossSpecialAttack1.json	2026/04/17 16:25	JSON ソース ファイル	2 KB
BossSpecialAttack2.json	2026/04/17 16:25	JSON ソース ファイル	2 KB
BossSpecialSmallAttack1.json	2026/04/17 16:25	JSON ソース ファイル	2 KB

ボスの種類ごとに攻撃の数や種類を分けたいときに、
攻撃の数に関しては、フォルダの中のjsonファイルの数分、攻撃が生成される仕組みになっています。
種類に関してはjsonファイルのデータをいじることにより、攻撃の種類を変えることが出来ます。

ボスのパラメーター

データの構想記録メモ

事前にどんなイベントが必要かをメモ帳に記録をしてからコードに記述

メモ帳に書いてあることをすべて変数にして管理
使用するかどうかの変数を作って判定

投擲イベント

- 投擲を行うかどうか
- 投擲物ごとの詳細データ
- モーション（腕など）と連動するかどうか
- 投擲タイミングのフレーム指定
- 投擲開始タイミング
- オブジェクトの出現タイミング
- 投擲する個数
- 出現位置
- 投擲の間隔時間
- 間隔を変動させるかどうか
- 間隔の最大値・最小値

移動イベント（前進）

- 前進移動を行うかどうか
- 移動速度

プレイヤー追従イベント

- プレイヤー追従を行うかどうか
- 基本となる移動速度
- プレイヤー接近時に停止するかどうか
- 停止可能かどうかの条件フラグ
- 既存速度に加算するかどうか
- プレイヤーとの基準距離
- 移動速度の最大値・最小値
- 追従の開始および終了タイミング

攻撃のデータの中身は、データを管理しやすいように**イベント**を作成し、管理をしています。
必要なイベントだけを選択をし、組み合わせることによって攻撃の動きを生成しています。
これによってパラメーターを入れるだけで、ボスの攻撃の動きの作成が可能になりました。

ボスの攻撃の作成 & 調整ツール

だけど

Jsonデータ化をして簡単に調整を出来るようになった。**クラスを一つ**にまとめられることが出来た

新しい攻撃を作るのが大変
どのパラメーターがどのイベントに対応してるか分からない。

攻撃ごとに.cpp.hを作成してた

```
▷ ⚙️ ++ BossAttackBase.cpp
▷ ⚙️ Ⓜ️ BossAttackBase.h
▷ ⚙️ ++ BossNormalAttack1.cpp
▷ ⚙️ Ⓜ️ BossNormalAttack1.h
▷ ⚙️ ++ BossNormalAttack2.cpp
▷ ⚙️ Ⓜ️ BossNormalAttack2.h
▷ ⚙️ ++ BossNormalAttack3.cpp
▷ ⚙️ Ⓜ️ BossNormalAttack3.h
▷ ⚙️ ++ BossNormalAttack4.cpp
▷ ⚙️ Ⓜ️ BossNormalAttack4.h
▷ ⚙️ ++ BossNormalAttack5.cpp
▷ ⚙️ Ⓜ️ BossNormalAttack5.h
```

```
▷ ⚙️ ++ BossNormalAttack6.cpp
▷ ⚙️ Ⓜ️ BossNormalAttack6.h
▷ ⚙️ ++ BossSpecialAttack1.cpp
▷ ⚙️ Ⓜ️ BossSpecialAttack1.h
▷ ⚙️ ++ BossSpecialAttack2.cpp
▷ ⚙️ Ⓜ️ BossSpecialAttack2.h
▷ ⚙️ ++ BossSpecialSmallAttack1.
▷ ⚙️ Ⓜ️ BossSpecialSmallAttack1.
```

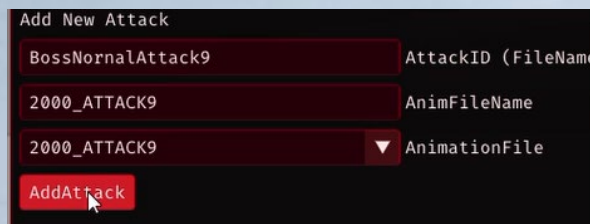
一つのクラスに攻撃をまとめた

```
Attack
▷ ⚙️ ++ BossAttackBase.cpp
▷ ⚙️ Ⓜ️ BossAttackBase.h
▷ ⚙️ Ⓜ️ BossAttackJsonParse.h
```

Jsonファイル化をすることによって調整をしやすくなったが、新規で攻撃を作る際にすごく大変になってしまったと感じたので
ゲーム内にツールを作成し、ゲーム内で攻撃を生成 & 調整を出来るようにしました。

ボスの攻撃の作成 & 調整ツール

攻撃の作成



Add New Attack

BossNornalAttack9 AttackID (FileName)

2000_ATTACK9 AnimFileName

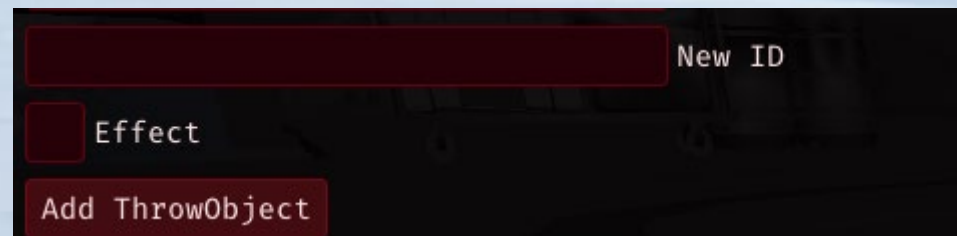
2000_ATTACK9 AnimationFile

AddAttack

攻撃のID
アニメーションファイル名
を書き込むとモーションデータ以外は空っぽの攻撃が生成

アニメーションファイル名は入力以外にリストから選べるようにしている。

投擲物の作成



New ID

Effect

Add ThrowObject

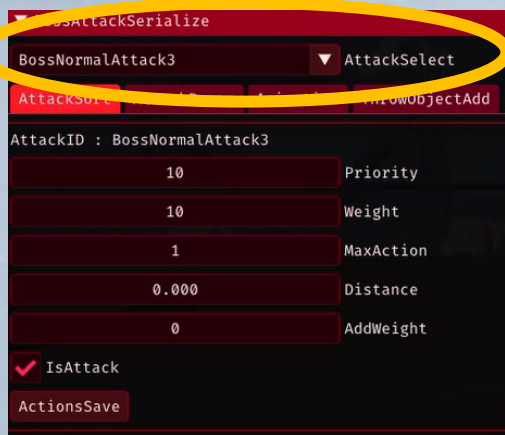
モデルファイル名
を書き込むと投擲物のデータを生成

投擲物はモデルデータだけでなくエフェクトにも対応している。
チェックボックスをONにするとエフェクトになる。

データの動的な追加に関しましては
攻撃の追加、投擲物の生成が出来るようになっています。
Addボタンを押すと攻撃が生成されjsonファイルが生成されます。

ボスの攻撃の作成 & 調整ツール

攻撃振り分けパラメーターの調整



The screenshot shows the 'BossAttackSerialize' window. At the top, there is a dropdown menu labeled 'AttackSelect' which is currently set to 'BossNormalAttack3'. This dropdown is highlighted with a yellow circle. Below it, there are several input fields for parameters: 'Priority' (10), 'Weight' (10), 'MaxAction' (1), 'Distance' (0.000), and 'AddWeight' (0). There is also a checkbox for 'IsAttack' which is checked, and a button labeled 'ActionsSave'.

攻撃情報の調整



The screenshot shows the 'Basic' panel for adjusting attack information. It contains a list of parameters with input fields and labels: 'HitDamage' (0.000), 'SlowTime' (0.000), 'SlowAmount' (0.000), 'SpeedUp' (0.000), 'AttackStart' (0.000), 'AttackEnd' (0.000), 'JustAvoidStart' (0.000), 'JustAvoidEnd' (0.000), 'AttackFrame' (0), 'SoundTime' (0.000), 'AttackCollTransform' (expanded), 'JustAvoidTransform' (expanded), 'UseFlash' (checkbox), and 'FlashStartTime' (0.000).

アニメーションデータの調整



The screenshot shows the animation data adjustment panel. It includes fields for 'Anim' (2000_ATTACK9), 'ModelHandle' (Loop), 'MaxFrame' (146.000), 'PlaySpeed' (1.000), 'EventStart' (-1.000), 'EventEnd' (-1.000), and 'FileName' (2000_ATTACK9). There is also a button labeled 'AnimSave'.

この丸のところで攻撃の選択

攻撃を選択、または生成をしてから上記3つのデータを調整する

調整に関しましては、攻撃の振り分けパラメーターの調整、攻撃に使うアニメーションデータの調整、投擲物の調整が出来るようになっています。
Saveボタンを押すとjsonファイルの書き換えをします。

ボスの攻撃の作成 & 調整ツール

投擲攻撃



ジャンプ攻撃



ステージ0のボスはもともとあった
のを**データ化**をして作成
ステージ1のボスは**ツール**のみで作
成
といった感じで攻撃を作成しました

以上のツールを使うことによってソースファイルをいじらず、かつゲーム内で攻撃の生成とパラメーターの調整が出来るようになりました。

カットシーンのパラメーター

項目一覧

- 到着地点
- 開始地点
- 注視点のオフセット
- 時間
- Easingの種類
- 最初の基準点
- 追従するオブジェクト

が右のJsonファイルに格納されています。

Jsonファイル

```
cutScene0 : {  
  "camera": {  
    "endPos": { // 到着地点  
      "x": -1000.0,  
      "y": 0.0,  
      "z": 1000.0  
    },  
    "startPos": { // 開始地点  
      "x": 1000.0,  
      "y": 0.0,  
      "z": 1000.0  
    },  
    "target": { // 注視点のオフセット  
      "x": 0.0,  
      "y": 400.0,  
      "z": 0.0  
    }  
  },  
  "duration": 1.0, // 何秒で終了地点に達するか  
  "ease": "InOut", // Easingのタイプの種類  
  "firstPosBaseName": "World", // 最初の基準点  
  "followPos": "Player", // どのオブジェクトに追従するか(座標)  
  "followTarget": "Player" // どのオブジェクトに追従するか(注視点)  
}
```

それらのデータをJsonデータで管理することによってソースファイルを減らし、一つのクラスで管理できるようになりました。
またカットシーンをjsonデータをいじらず、ゲーム内で作成出来るようにツールを作成しました。

カットシーン制作ツール

カットシーン制作ツール

勝利演出カメラ



ボス登場前カメラ



プレイ開始前カメラ



演出用のカメラを一つ一つに対して、.cpp.hを用意して管理するのはすごくコストがかかってしまうと感じたのでカットシーン作成ツールを作成しました。

カットシーン制作ツール

機能の画像

プレビュー機能



ファイル名を打ち込んで
ロードが出来る機能



誤セーブを防ぐ機能



すでにあるファイル名を打ち込むと
StartPos、EndPosがロードされ反映される

使われているファイルをセーブ
するときにPopUpが出る。

使いやすくするために様々な機能を作成しており、
プレビュー機能、ファイル名を入力したらロードが出来る機能、誤セーブを防ぐ機能。
などを追加して、より使いやすくしました。

カットシーン制作ツール

設定画面

座標や注視点の設定

▼ Target

0.000	0.000	0.000	Base Target
0.000	0.000	0.000	Target Offset
Player	PlayerOffsetTarget	Enemy	EnemyOffsetTarget
World			

▼ Follow Position

300.000	30.501	-2000.000	Base Position
0.000	0.000	0.000	Offset
Player	FirstPosPlayer	Enemy	FirstPosEnemy
World			

Playerのボタンを押すとplayerに追従する

FirstPosPlayerのボタンを押すとPlayerを基準点にして開始される。
(追従はしない)

開始の位置と終了の位置と時間の設定

0.000	0.000	0.000	Start Pos	Set Start
Player	Enemy	World		
0.000	0.000	0.000	End Pos	Set End
0.000			Duration	

Set StartやSet Endボタンを押すと今のカメラの位置が開始位置、または終了位置に設定される

Easingの種類設定

▼ Easing

Linear ▼ Easing

Linear

EaseIn

EaseOut

EaseInOut

Easingの設定はドロップダウンメニューにして表示

気持ちよさの演出

気持ちよさの演出

ジャスト回避時の演出

スクリーンエフェクト



モーションブラー



ジャスト回避時に

世界のスロー、スクリーンエフェクト、BGMを止める、モーションブラー
を実装していき、回避のアクションが気持ちよくなり、回避の楽しさが上がりました。

気持ちよさの演出

ガウスフィルタあり



ガウスフィルタなし



さらに、スクリーンエフェクトには**ガウスフィルタ**による画面ぼかしを入れることによって、よりジャスト回避感を強めることが出来るようになりました。

気持ちよさの演出

攻撃時の演出



ダメージフラッシュで当たった敵は**赤色**に、斬撃エフェクトはプレイヤーが切り刻んだ向きになるように調節をしている

プレイヤーの攻撃時には
ヒットストップ、コントローラー振動、画面振動、ダメージ表記、ダメージフラッシュなどを実装していき、攻撃時に気持ちよくさせることが出来ました。

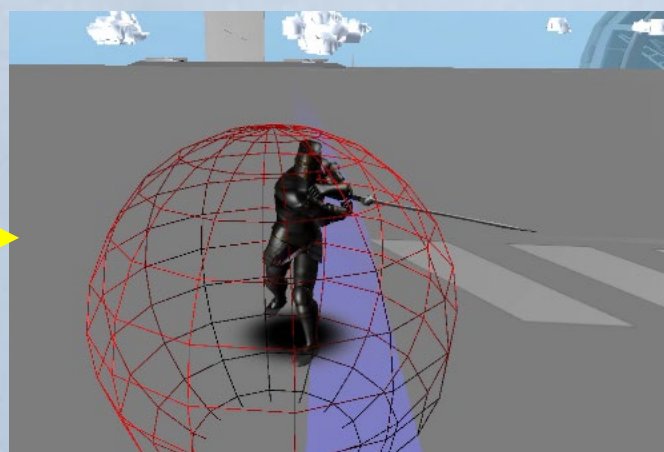
気持ちよさの演出

攻撃時の当たり判定について

開始



攻撃判定



攻撃終わり



当たり判定生成に関してはプレイヤーが剣を振っているタイミングで取るようにモーションの再生フレーム数で判定の開始、終了を判定しています。

ボスの攻撃時における工夫

ボスの攻撃時における工夫

このままだとやりにくかったので



改善をしていきました



ジャスト回避を追加した時にはジャスト回避がしにくく、とても難易度が高かったので回避をしやすいように敵の攻撃に工夫を入れました。

ボスの攻撃時における工夫

攻撃する部位を光らせる

攻撃が来る前に攻撃の
モーション速度を一瞬落
とす



音を鳴らす

- ・ユーザーがジャスト回避をしやすくするために、**視覚、聴覚、モーションスピード調整**の三点に意識をして制作をしていきました。
- ・これらを機能を実装していくことによりジャスト回避がとてもしやすくなり、展示会の際に初めて触った人にも回避が出来るような仕組みに出来ました。

ボスの攻撃時における工夫

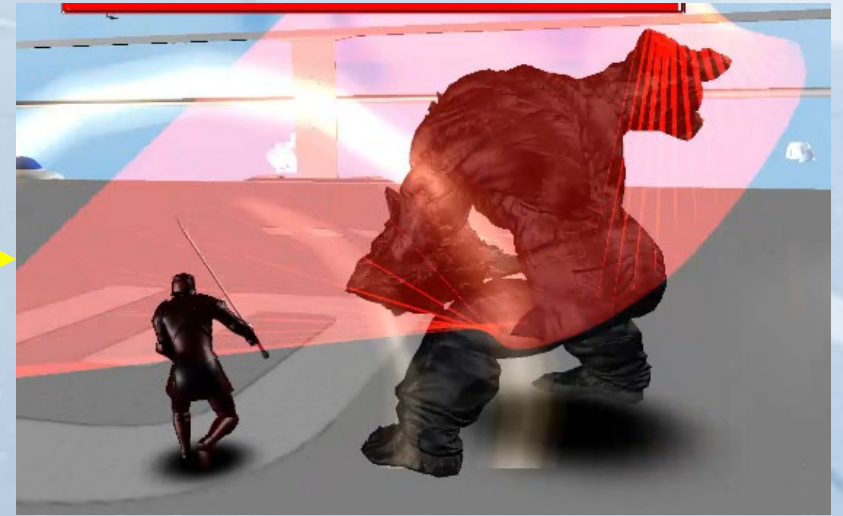
攻撃始まり



プレイヤーのいる
向きに回転



攻撃終わり



また、ボスの攻撃を普通に避けられてしまうのを避けるためにボスの攻撃の時にプレイヤーの方向に向かせてジャスト回避をしやすくしました。

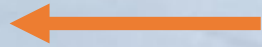


ロックオンカメラ

ロックオンカメラ

右スティックを右側に倒す

右スティックを左側に倒す



左側の敵にロック

右側の敵にロック



- ・プレイヤーのカメラ操作を極力減らすためにロックオンカメラを実装しました。
- ・また多人数の敵に対しても自然にロックオンカメラロックオンの仕組みを作成しました。右スティックを右側に倒すと右側に、左側に倒すと左側にターゲット変更が出来ます。

ロックオンカメラ

- ・ 画面の正面にいるか
- ・ 入力した方向（右／左）にいるか
- ・ 現在の注目対象から離れすぎていないか

ロックオンするターゲットの決定方法は評価値をもとに行われています。
評価値は上記項目になっています。

ロックオンカメラ

評価値高い

正面判定の内積の式 : (frontDotが1に近づけばより正面)

```
//正面角度の反映  
float frontDot = VDot(camFront.Normalize(), dotEnemyPos.Normalize());  
//正面角度のスコアの計算  
float frontScore = (1.0f - frontDot) * 1.0f;
```

frontScoreが0に近いほど、評価値は高くなる。
より正面なら評価値は高くなる。

- ・ 正面判定と左右判定は内積を使ってやっており、プレイヤーの左右ベクトルと敵とプレイヤーのベクトルで判定をしています。
- ・ また角度成分だけを取りたいので0~1にの範囲に抑えるためにベクトルを正規化して計算をしています。
- ・ 左右判定もこの内積の計算方法で計算をしています。

0

0

-1

評価値低い



ロックオンカメラ

敵とプレイヤーの距離の関係の図

距離の長さの割合を求める式
距離が近いと評価値が高い

```
//距離の遠さの反映  
float dist = VECTOR3(camPos - targetEnemyPos).Size();  
//距離のスコアの計算  
float distScore = (dist / CAM_LONG_DISTANCE) * 0.3f;
```



また距離判定についてはプレイヤーと敵の距離を基準の距離をもとに割合を使い算出しています。

ロックオンカメラ

$$\begin{aligned} & \text{正面判定の評価値} \times 1.0 \\ & + \\ & \text{左右判定の評価値} \times 0.5 \\ & + \\ & \text{距離判定の評価値} \times 0.3 \end{aligned}$$

最終的な評価値は上記の計算式を合算した式を元に算出されており、基本的には評価値が低い敵に対してロックオンをします。

ロックオンカメラ

評価値が一番**高い敵**をロックオンするパターン

右スティックを右側に倒す



一番左の敵に移動



右スティックを左側に倒す



一番右側の敵に移動



これ以上**左に敵がい**ないときに左側にスティックを傾けた時に**一番右の敵**に行くようにしているのですが、その時は右側に対して評価値が一番高い敵に向くようにしています。

基本設計

基本設計

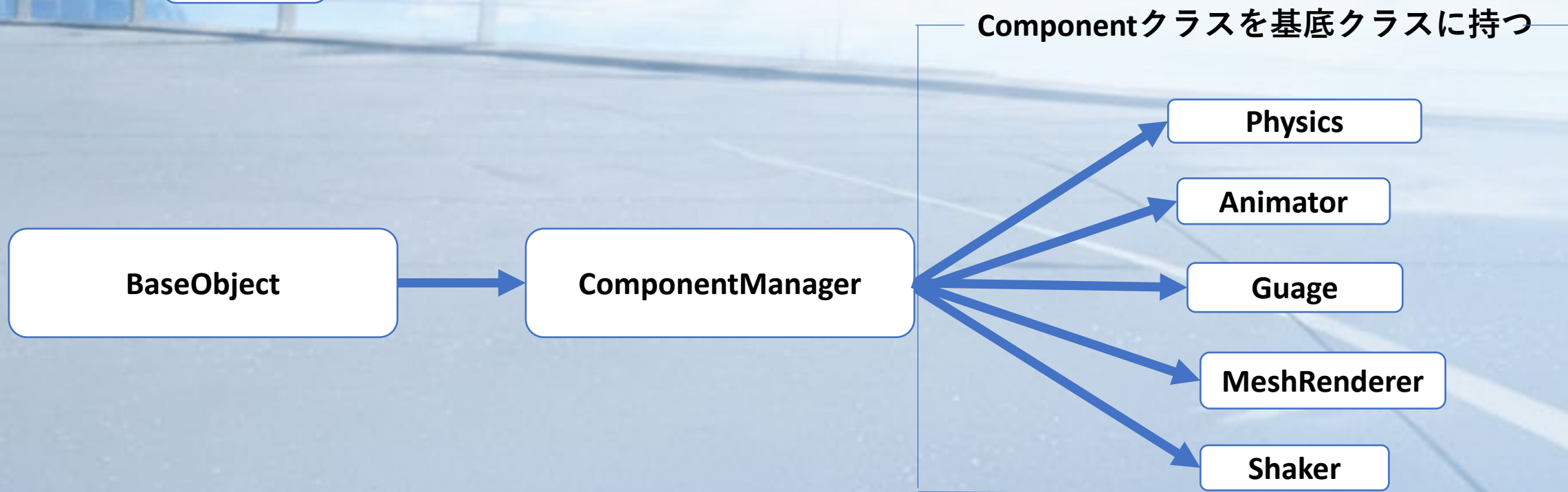
私が全体設計で意識をしたことは

- ・ コンポーネント指向による、オブジェクトのコンポーネント化
 - ・ ステートパターンの実装
- の2つに注力をして実装をしていきました。

所有

クラス

コンポーネント化



などのクラスをくっつける

どのオブジェクトでも汎用的に使えるように、値さえ変えてしまえば処理の内容自体を変えなくてもいいようにコンポーネント化をしていきました。

コンポーネントには基底クラスにComponentクラスを持っており、ComponentManagerクラスで管理をされるような設計にしました。

ステートパターン

ステートの基底クラス

```
class StateBase
{
public:
    StateBase() {
        id = StateID::STATE_MAX;
        string = "";
        obj = nullptr;
        com = nullptr;
    }
    virtual ~StateBase() {}

    void Init(BaseObject* _base, StateID::State_ID _id) {
        obj = _base;
        id = _id;
    }
}
```

ステートを生成する関数

```
//ステートを追加
template <typename T>
std::shared_ptr<T> CreateState(std::string _state, StateID::State_ID _id) {
    std::shared_ptr<T> sta = std::make_shared<T>();
    StateID::SetID(_state, _id);
    sta->Init(obj, _id);
    stateInfo.emplace(_state, sta);
    return sta;
}
```

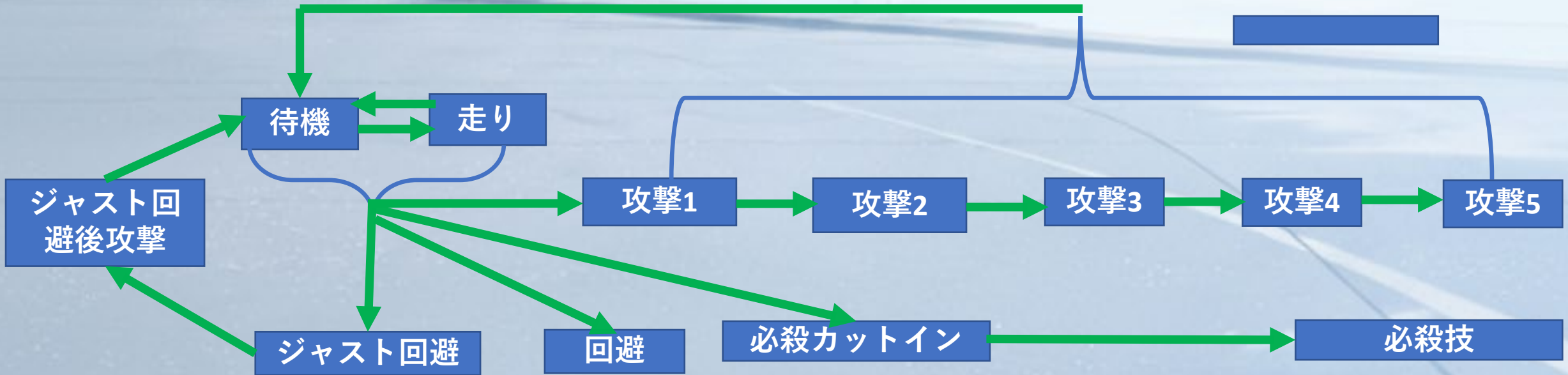
プレイヤー、カメラ、敵全体の状態をステートパターンで実装していききました。
すべてのオブジェクトで汎用的に使用できるようにするStateManagerやStateBaseを作成しました。
CreateState関数呼んでステートが簡単に生成できるようになっています。

プレイヤーの状態遷移図

状態の流れ



クラス



プレイヤーの状態遷移図です。
これら一つ一つをクラス化して、一つのファイルに処理をため込みすぎないようにしています。

今後の展望

今後の展望

今後の展望といたしましては

- ・ **ボスと雑魚敵の連携、(ボス戦に雑魚敵を出す)**
- ・ **ボスの攻撃のバリエーションを増やす**

などといったことをしていきたいと考えています。

そうすることによってゲームの完成度を高めて、より、面白いゲームを作れるようなプログラマーになっていきたいです。



最後に

最後に

ブースの様子



メンバーと写真



本作品は「ゲームクリエイター甲子園 & 神ゲーエボリューション合同作品展示会」に出展を
しており、たくさんの人に評価や感想をもらうことが出来て、一般ユーザーに遊んでもらった
時にどういう動きになるかを見て、操作感や、回避のしやすさを調整していきました。

最後に

ゲームポスター



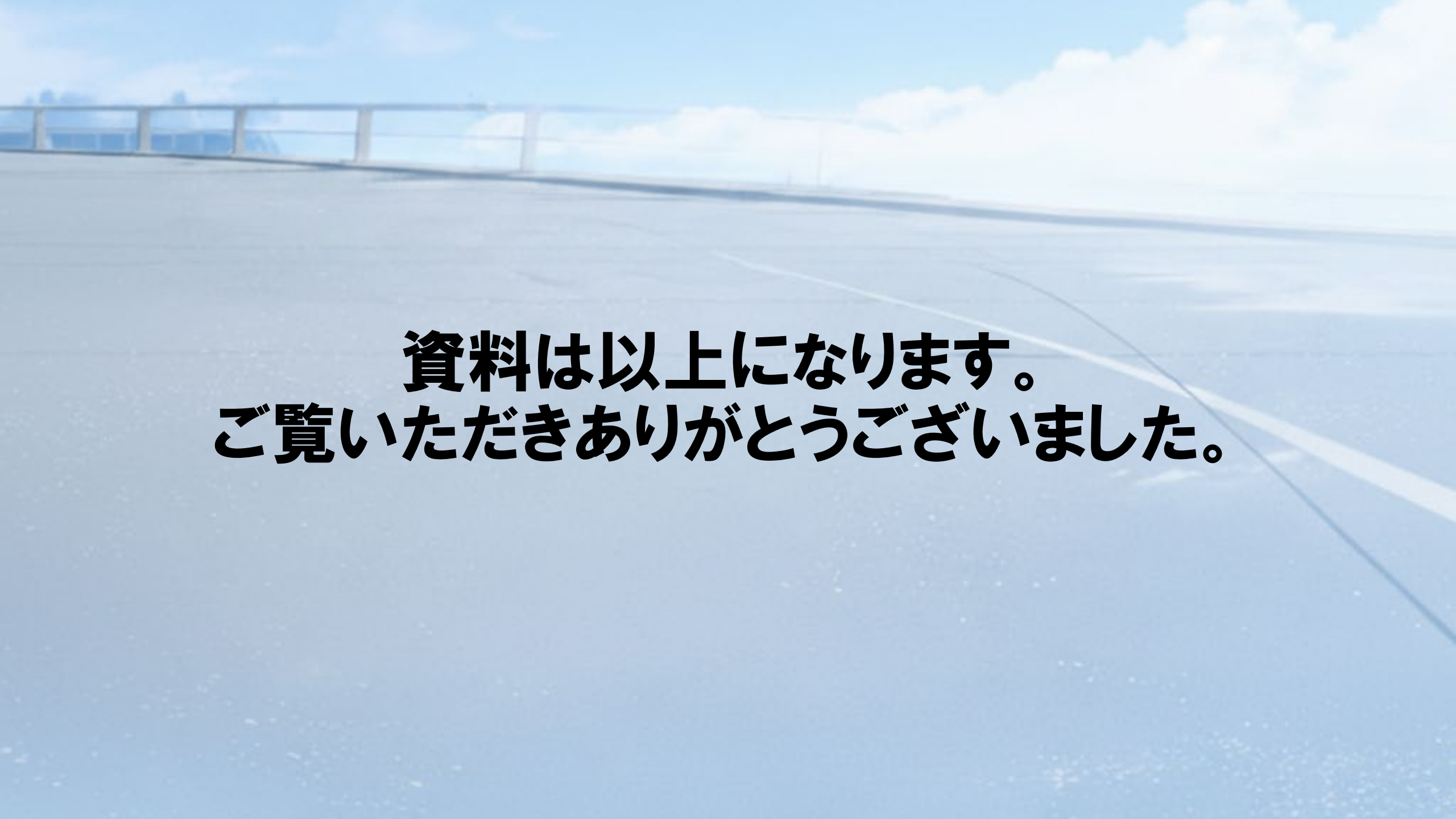
ゲームの写真



ブースの様子



また過去にも作品を「学生インディーゲーム展」に出展しており、こちらの作品は総合大賞ノミネートされており、一定以上の評価をしてもらっている作品となっています。
グラバライドの作品URL: <https://gameparade.creators-guild.com/works/2965>



**資料は以上になります。
ご覧いただきありがとうございました。**